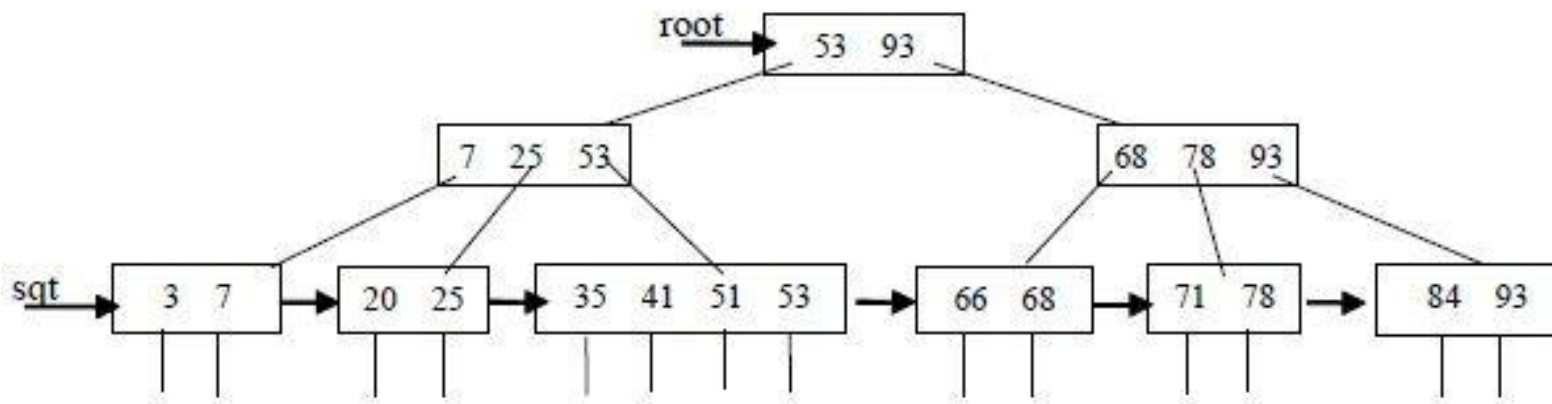


现代处理器设计

为什么要学硬件？

- **抽象!= 实现**
 - 问题：什么样的B+树更好？
 - 在DRAM和NVM上的区别？
 - 针对NVM硬件特性可做哪些优化？



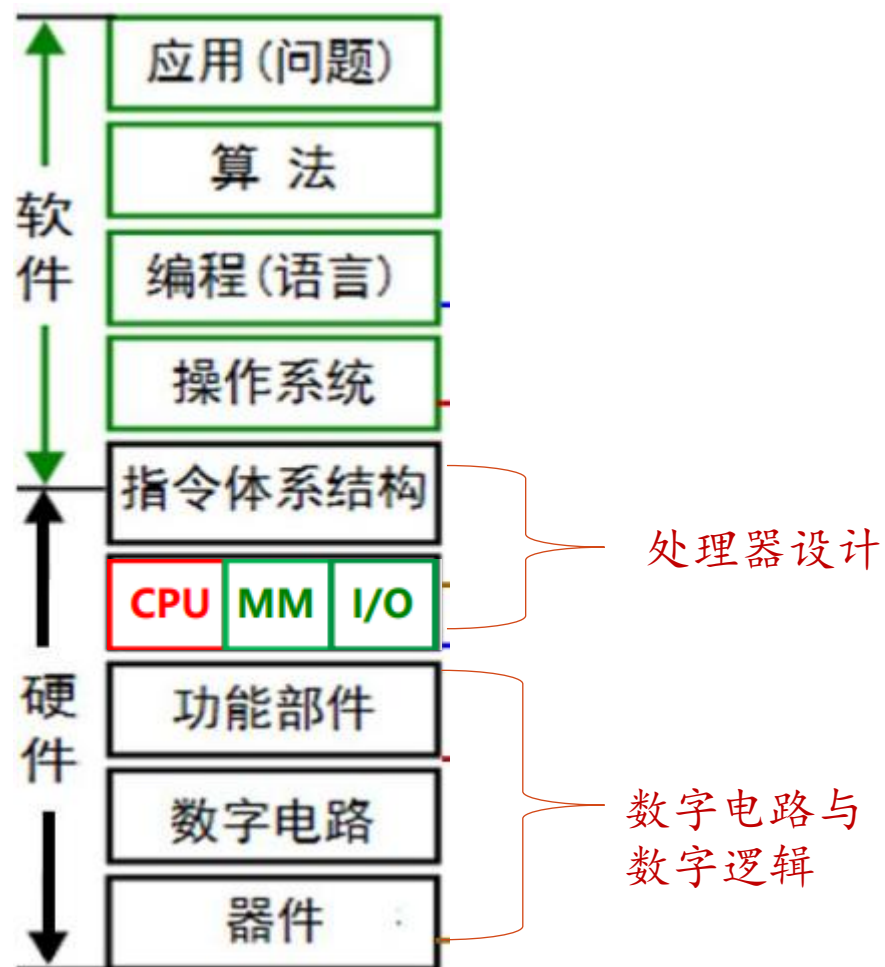
为什么要学硬件？

- 软件vs.硬件：根据性能、成本、功耗、灵活性权衡的动态边界
 - 硬件功能上移到软件：灵活，例如
 - 软件定义网络（SDN）：路由策略由软件控制，硬件只负责转发。
 - 软件定义存储（SDS）：RAID、快照、压缩等功能由软件栈实现。
 - 软件功能下沉到硬件：高效，例如
 - GPU / NPU / TPU：专用硬件支持高度并行计算，但需特定编程模型。
 - FPGA：可编程的逻辑电路，实现定制加速，可用于通信领域的数据高速传输，数学计算（金融数据，医学数据分析）。
 - NVM上的COW B+树：将数据库索引更新逻辑放到NVM内存控制器，实现近数据处理。
- 软件与硬件的协同优化
 - RDMA：绕过CPU和操作系统，直接内存访问。
 - OpenCL：（Open Computing Language）：异构并行编程的标准框架
 - 支持多厂商硬件（如Intel CPU、NVIDIA/AMD GPU、ARM芯片）

为什么要学硬件？

- 我们正处于后PC时代
 - 前端：个人移动设备（PMD）取代PC
 - 能耗，计算速度，存储
 - 后端：云计算/巨型计算中心(仓库规模计算机)
 - 实现软件即服务（Software as a Service,SaaS）
- 新技术（自动驾驶...），新硬件（GPU，...）
- 当前时代背景

课程在计算机专业课的位置



课程内容

- 1. 计算机系统概述
- 2. 数字电路基础
- 3. 指令与汇编语言
- 4. CPU原理与设计
- 5. 计算机的算术运算
- 6. 存储器

课程目标

- “努力一学期，设计个处理器”
 - MIPS 32位CPU（RISC架构）
 - MIPS 架构简洁、规整
 - 它体现了 RISC 的原始设计理念
 - 许多概念（如 TLB 管理、异常处理）在 ARM、RISC-V 中依然适用
 - 支持几十条汇编指令：工业标准
 - 参照全国比赛标准
 - 全国大学生系统能力培养大赛

全国大学生系统能力培养竞赛

- **“龙芯杯” 全国大学生计算机系统能力培养大赛（MIPS赛道）**
 - www.nscscc.com 2017-2025
 - 开发支持MIPS基准指令集的微系统
 - 功能测试：MIPS-C指令集
 - 性能测试
 - 流水线、超标量、预测、Cache等提升CPU性能

实验安排计划

- 1. 一位/四位全加器设计
- 2. 4位超前进位加法器设计
- 3. 32位ALU模块设计
- 4. 通用寄存器堆, 计数器PC, 指令内存、数据内存设计
- 5. 汇编程序编写和验证
- 6. 单周期CPU设计, 10条指令
- 7. 多周期流水线CPU设计, 10条指令
- 8. 多周期流水线CPU设计, 50条指令 (期末测试)

参考教材

- 计算机组成与设计(--硬件/软件接口)

- 原书第5版：MIPS版
- Berkeley&Stanford

- 参考资料

- See MIPS Run Linux

See MIPS Run Linux (2nd edition)

看 MIPS 跑 Linux （第二版）

——MIPS 体系结构剖析

D. Sweetman（斯维特曼）著

J.Qu（屈建勤）译

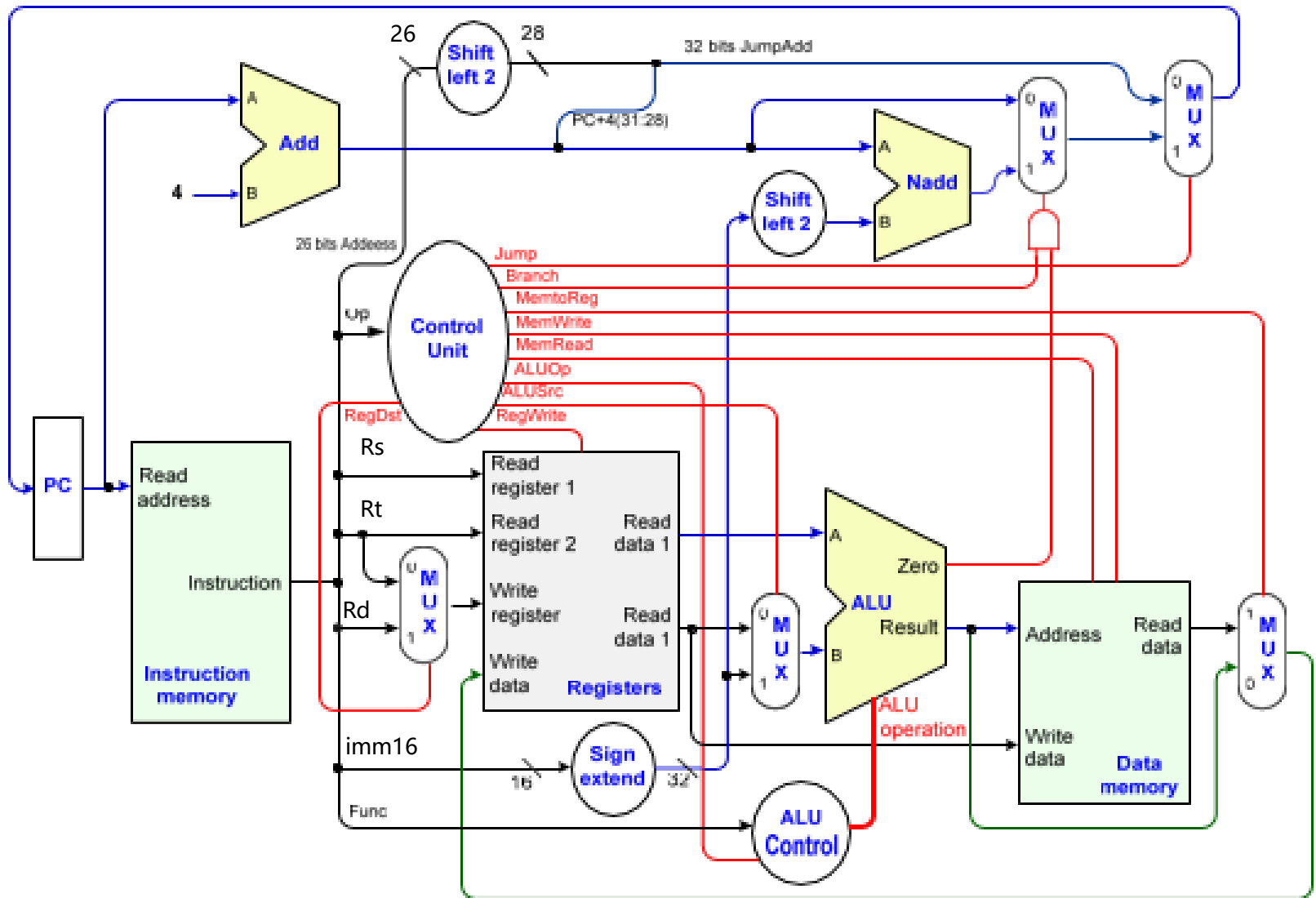
2007 年 12 月 31 日



本课程的考核

- 课堂表现+作业+随堂小测：35%
- 实验（1-7）：35%
- 期末考试（实验8）：30%

单周期控制器设计（包含跳转指令的数据通路）



第1章 计算机系统概述

- **1.1 概述**
- **1.2 计算体现结构中的8个伟大思想**
- **1.3 计算机的发展**
- **1.4 计算机中的程序**
- **1.5 性能评价**
- **1.6 功耗与成本**

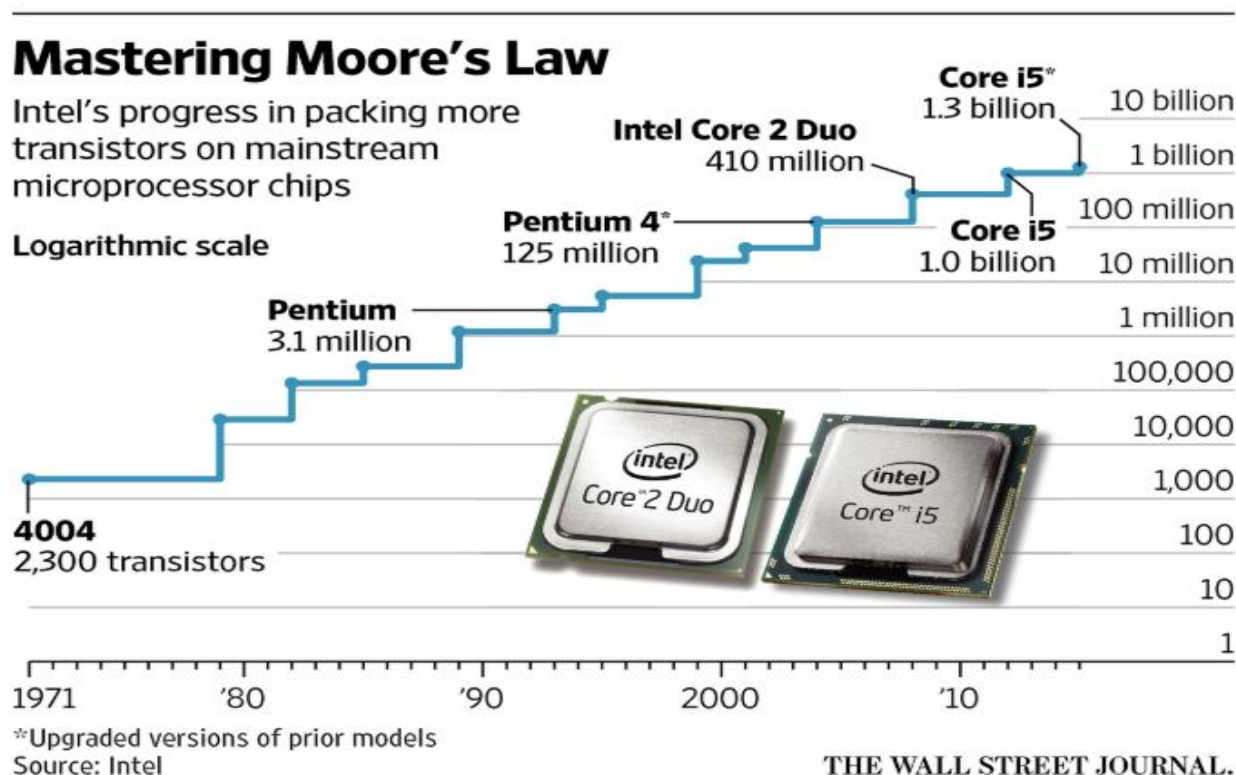
1.1 概述

- 计算机系统目标: 更快、更大、更强:
 - 50-70年代: 做出可用、更快的计算机;
 - 80年代开始: 个人计算机普及;
 - 90年代开始: 互联网普及;
 - 2000年以后: 超大规模分布式系统
 - 2006年以后: 云计算和大数据时代, 后PC时代
- 计算机应用的分类:
 - 个人计算机
 - 服务器
 - 嵌入式计算机

1.2 计算机系统结构中的8个伟大思想

1. 面向摩尔定律的设计

- 1965 年，英特尔联合创始人戈登·摩尔提出以自己名字命名的「摩尔定律」：18-24个月芯片的集成度翻一番
- 目前，摩尔定律失效？



计算机系统结构中的8个伟大思想

2. 使用抽象简化设计：层次设计提高效率，高层看不到低层细节

- **抽象**：隐藏低层层次的实现细节，简化各层次上用户的使用，每一层次都为上一层隐藏了 自己的技术细节
- 计算机的层次
 - 外层是应用软件，底层是硬件，系统软件在两者之间
 - 系统软件：操作系统，编译程序，汇编程序



计算机系统结构中的8个伟大思想

3. 加速大概率事件更有利于提高性能

- 大概率事件：经常发生的事件，不同类型指令的使用频率不同

4. 通过并行提高性能

- 硬件性能的提高不再依赖更高的主频和CPI，而是借助并行

5. 通过流水线提高性能

- 充分利用CPU的各个部件

计算机系统结构中的8个伟大思想

6. 通过预测提高性能

- 条件结果未知前提前开始工作：如果预测相对准确，预测错误代价不高，例如分支语句的跳转

7. 存储器层次设计

- 从成本考虑，顶层存储器快但容量小，底层存储器慢但容量大

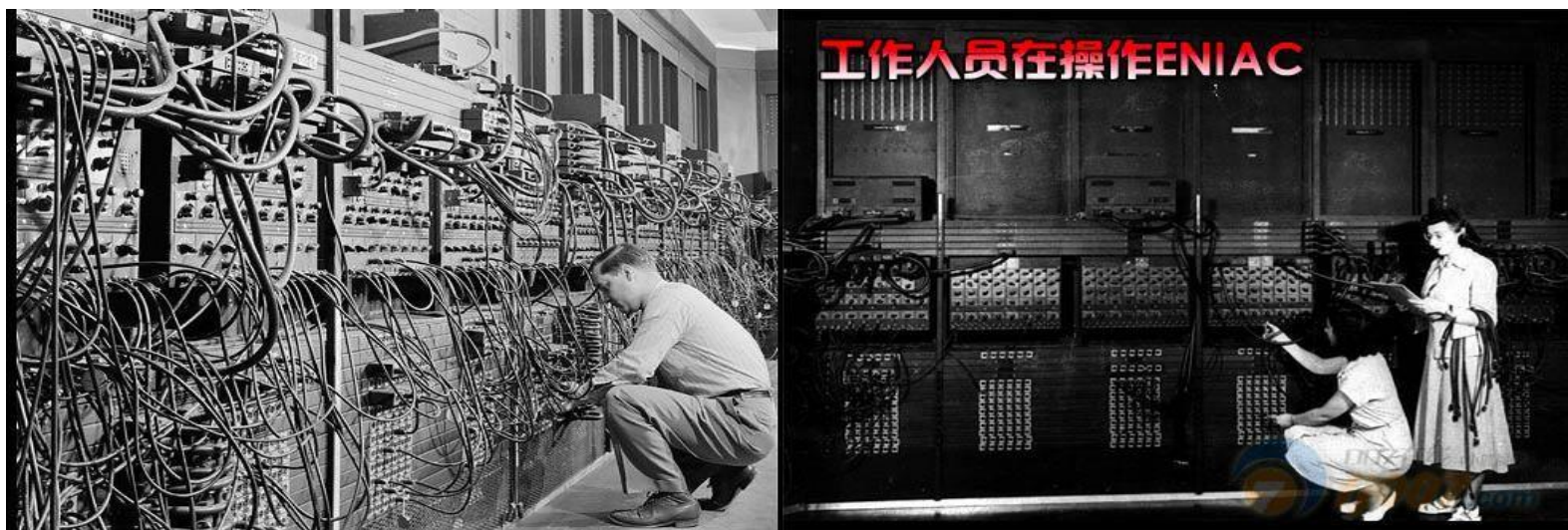
8. 通过冗余提高可靠性

- 引入冗余组件使系统更可靠

1.2 计算机的发展

1946年2月14日，第一台电子计算机ENIAC (Electronic Numerical Integrator And Computer, 电子数字积分计算机的简称) 诞生于美国宾夕法尼亚大学

机器	器件	加法运算速度	程序指令	用途
ENIAC	电子管	5000次/秒	硬件互连(连接线)编制程序	专用 (生产 火炮射表)



电子数字计算机ENIAC, 1946

计算机的发展历史

□ ENIAC的主要缺点之一：

- 利用开关和连接线来设置程序指令，程序设计麻烦，低效

□ ENIAC设计者的思考：

- 将程序指令存储在机器中，由它控制机器的操作？

□ 存储程序原理

- 冯·诺依曼第一次书面提出了存储程序数字计算机，包括运算器、控制器、存储器、输入输出装置，采用了二进制，并实现了内存存储程序，是目前一切电子计算机设计的基础。
- 英国剑桥大学于1949年最先制成了世界第一台用电子延迟存贮的程序内存电子计算机EDSAC。冯·诺依曼的EDVAC几经周折，在1952年终于制成。



计算机的发展历史

- 电子管时代（1945-1956）
 - 电子管作为基本电子元器件，体积大、耗电量大、寿命短、可靠性低、成本高
 - 没有系统软件，用机器语言和汇编语言编程
- 晶体管时代（1956-1963）
 - 主要元器件是晶体管
 - 使用高级语言（事务处理的COBOL语言、科学计算机用的ALGOL语言，和符号处理用的LISP），操作系统初步成型，计算机由手工操作改变为自动作业管理。

计算机的发展历史

- 电子计算机之集成电路时代（1963-1971）
 - 主要使用的元器件是中小规模集成电路
 - 1952年英国的Geoffrey Dummer（达默）首先提出了集成电路的设计思想
 - 1958年，美国德州仪器公司的工程师Kilby（基尔比）发明了集成电路，将三种电子元件结合到一片小小的硅片上，第一块集成电路（Integrated Circuit, IC）出现
- 电子计算机之超大规模集成电路时代（1971-）
 - 微处理器和各类计算机出现，性能迅速提高：字长4位、8位、16位、32位和64位的微型计算机相继问世和广泛应用。
 - 1981年，IBM推出个人计算机用于家庭、办公室和学校。
 - 1984年，苹果公司退出Macintosh系列个人电脑，提供了友好的图形界面，用户可以用鼠标方便地操作。

计算机的发展历史

- 架构分化与自动化设计（1980s-）
 - RISC与CISC之争
 - CISC（复杂指令集）：x86主导PC市场，通过复杂指令提升效率。
 - RISC（精简指令集）：MIPS, ARM陆续出现，低功耗、高并行性适配设备。
 - 设计方法革新：EDA工具链成熟，芯片被视为完整系统，引入高级建模与仿真
 - 性能提升技术：
 - 流水线、超标量架构：（如1993年Pentium）提升指令并行度
 - 缓存分层设计：（如1985年80386引入L1缓存）优化数据访问

计算机的发展历史

- 多核与异构计算时代（2000s-）
 - 多核架构普及：物理限制导致单核频率停滞，多核成为主流
 - 2001年，IBM的POWER4，2005年Intel/AMD推出双核CPU。
 - 技术支撑：SMP（对称多处理）、CMP（片上多核）架构。
 - 异构计算兴起
 - GPU通用化：NVIDIA CUDA（2006年）使GPU承担通用计算任务

计算机的发展历史

Computing Devices Now



Sensor Nets



Cameras



Media
Players



Set-top
boxes



Games



Laptops



Servers



Robots



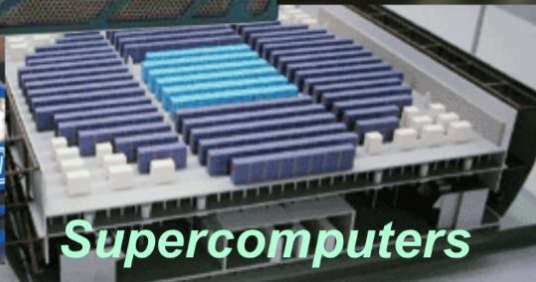
Smart
phones



Automobiles

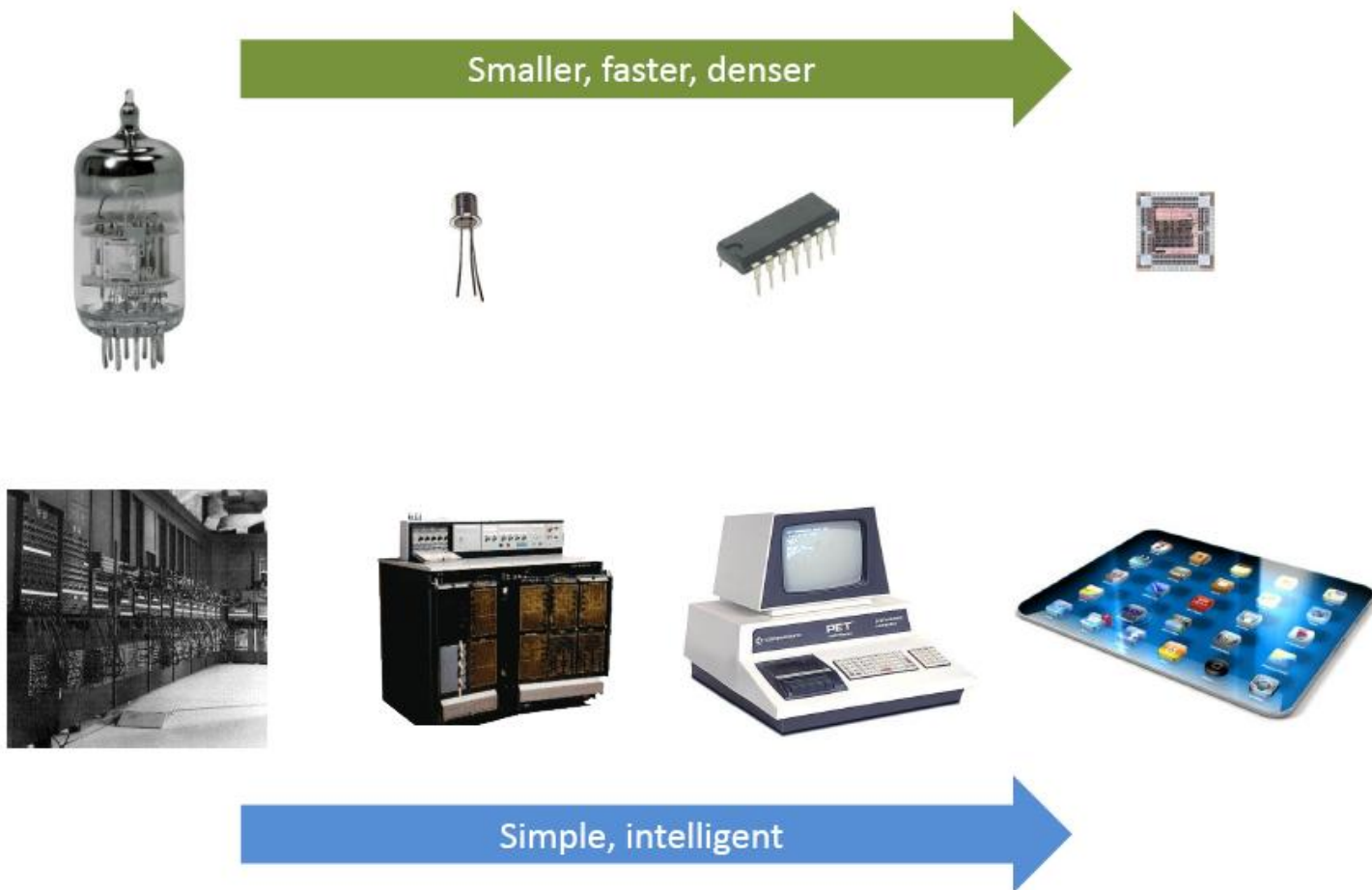


Routers

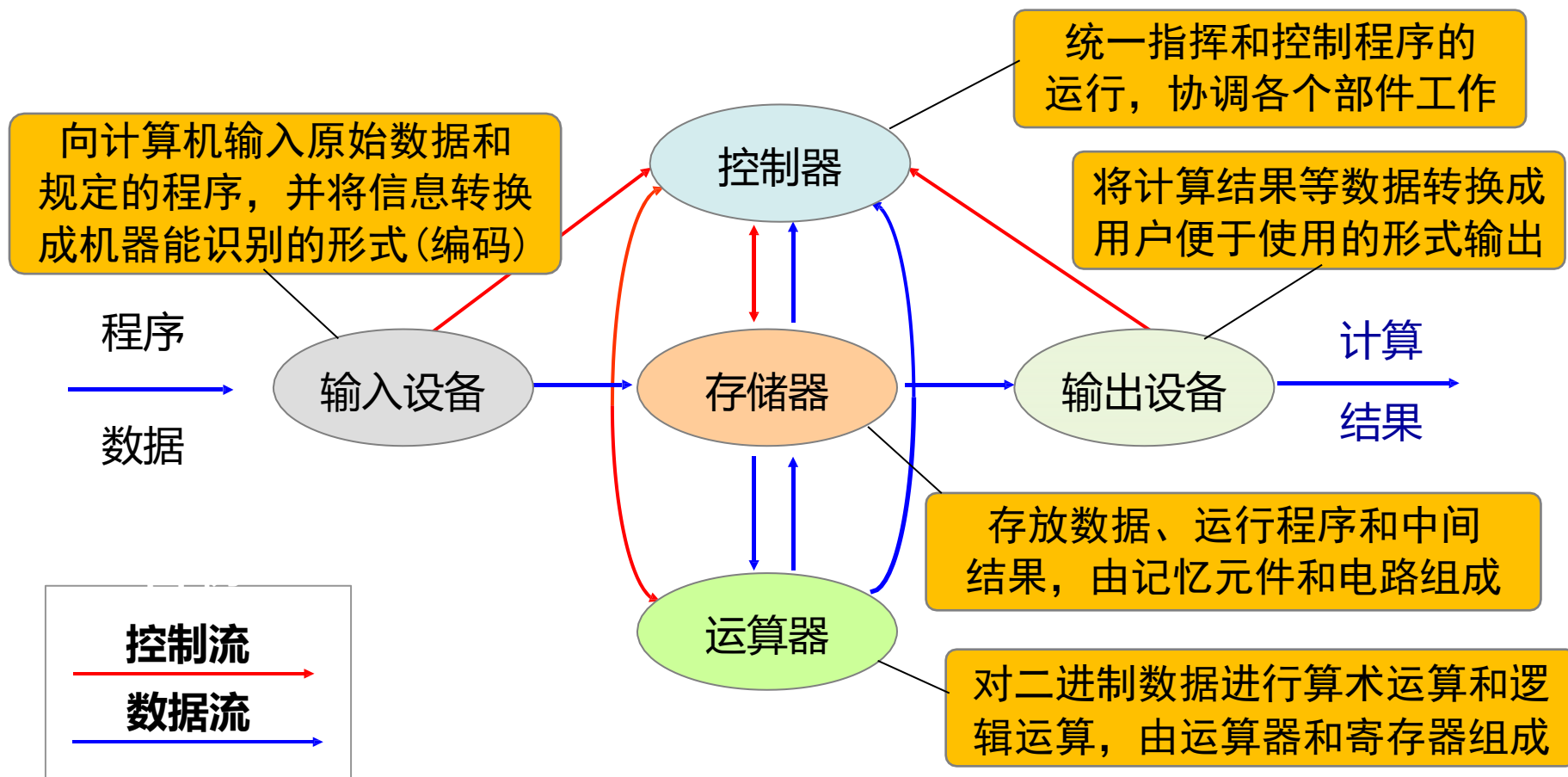


Supercomputers

计算机的发展历史



冯·诺依曼计算机框架



冯诺依曼体系结构

□ 功能部件1——处理器(CPU - Central Processing Unit)

- 功能：执行程序(Execute programs)

- 组成：控制器+ 数据通路

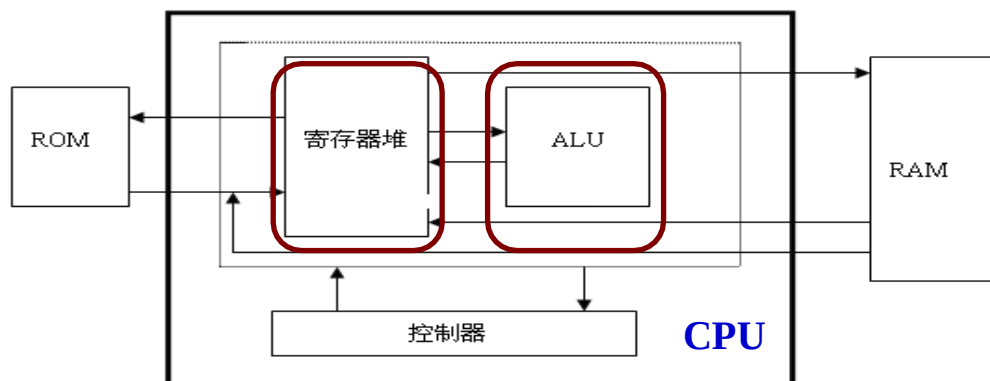
- 控制器：从存储器中读取计算机指令，然后解析指令，并按照指令的要求，指挥计算机的其他部件协调工作。

- 数据通路：完成指令的执行

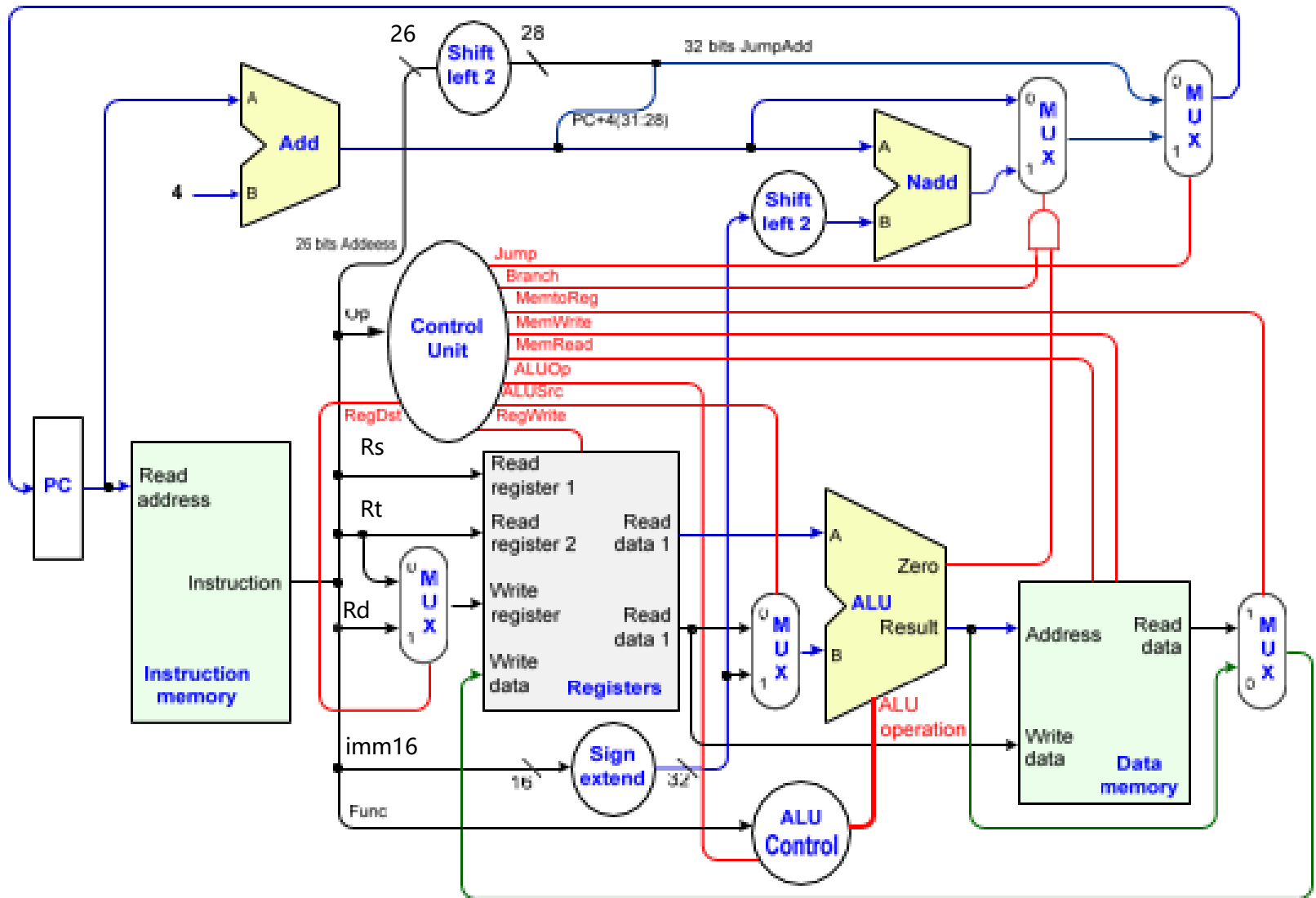
核心：ALU(Arithmetic Logic Unit)+Register(寄存器)

- ① ALU（运算器）：执行算术、逻辑运算

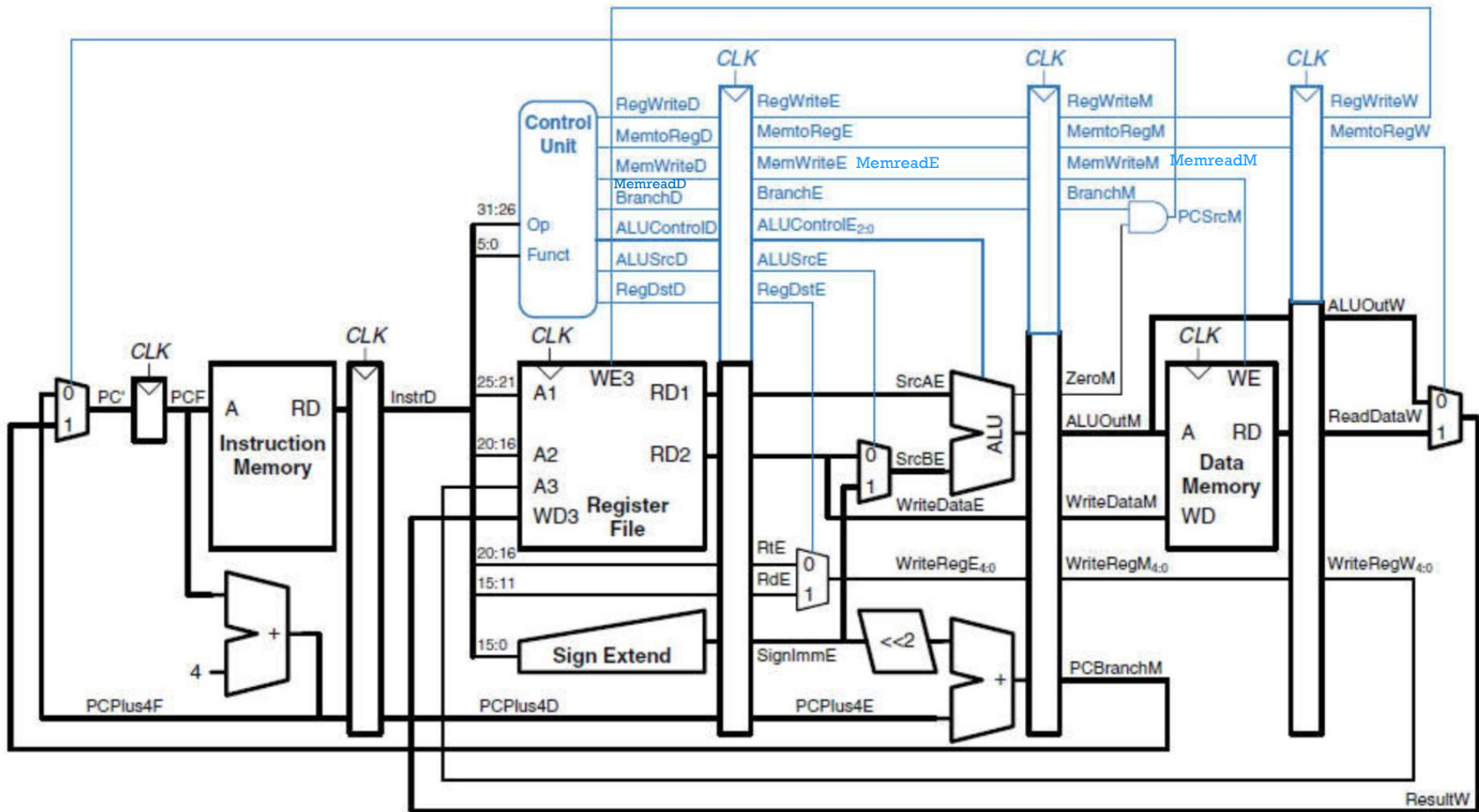
- ② 寄存器：存储临时数据或控制信息，例如：PC、IR等



单周期处理器设计



多周期处理器流水线设计



冯诺依曼体系结构

□ 功能部件2——存储器

- 功能：以二进制形式存储程序（即一系列计算机指令的集合）和数据。

- 层次化结构

- 内存：易失性存储器（volatile memory）

- 高速缓存 (Cache)：存放最近使用的数据和指令, 采用速度更快的静态随机访问存储器（SRAM）技术
 - 主存(Main Memory)：存放被启动的程序中的部分数据和指令，可以用动态随机访问存储器（DRAM）

- 外存：非易失性存储器（nonvolatile memory）

- 磁盘/NVM/闪存/固态盘：存放系统中所有软件和文档
 - 光盘(Optical disk)、磁带(Tape)：脱机存档



冯诺依曼体系结构

□ 功能部件3——输入/输出Input/Output (I/O)

- **功能:** 各种信息的输入/输出

- **组成:** I/O控制器+ I/O设备

- I/O控制器: 控制外设工作, 完成主机和外设 之间的通信

- I/O设备: 输入/输出信息

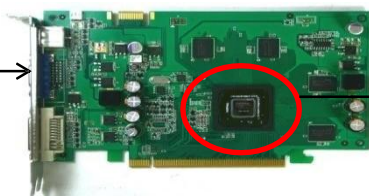
- 输入设备: 将信息输入计算机, 例如键盘、鼠标、扫描仪等;
 - 输出设备: 将计算结果或存储器中的内容从计算机输出出来, 例如显示器、打印机等。



透视计算机



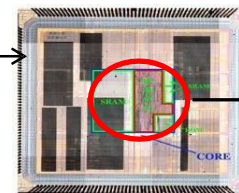
个人计算机由主机和
外设组成



主机中包含多个
电路板

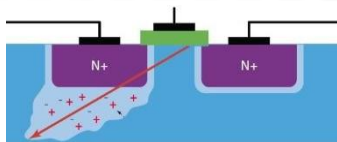


每个电路板中
有若干集成电路



每个集成电路
中有若干模块

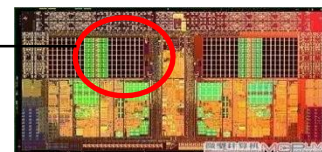
每个门电路实现基本的
逻辑运算



所有信息用二进制编
码表示



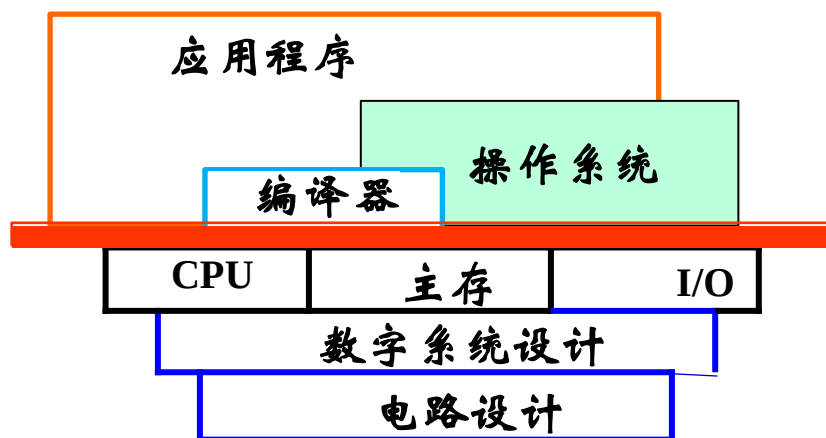
每个单元中
有若干门电路



每个模块中有上千
万个单元

1.4 计算机中的程序

□ 计算机的层次结构 (Computer Hierarchy)



- 计算机系统层次结构中，指令系统是软/硬件的交界面
- 不同用户工作在不同层次，看到的计算机是不一样的

从高级语言到硬件语言

例：C语言代码

```
Swap ( int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

编译

```
multi $2, $5, 4
add  $2, $4, $2
lw   $15, 0($2)
lw   $16, 4($2)
sw   $16, 0($2)
sw   $15, 4($2)
jr   $31
```

编译

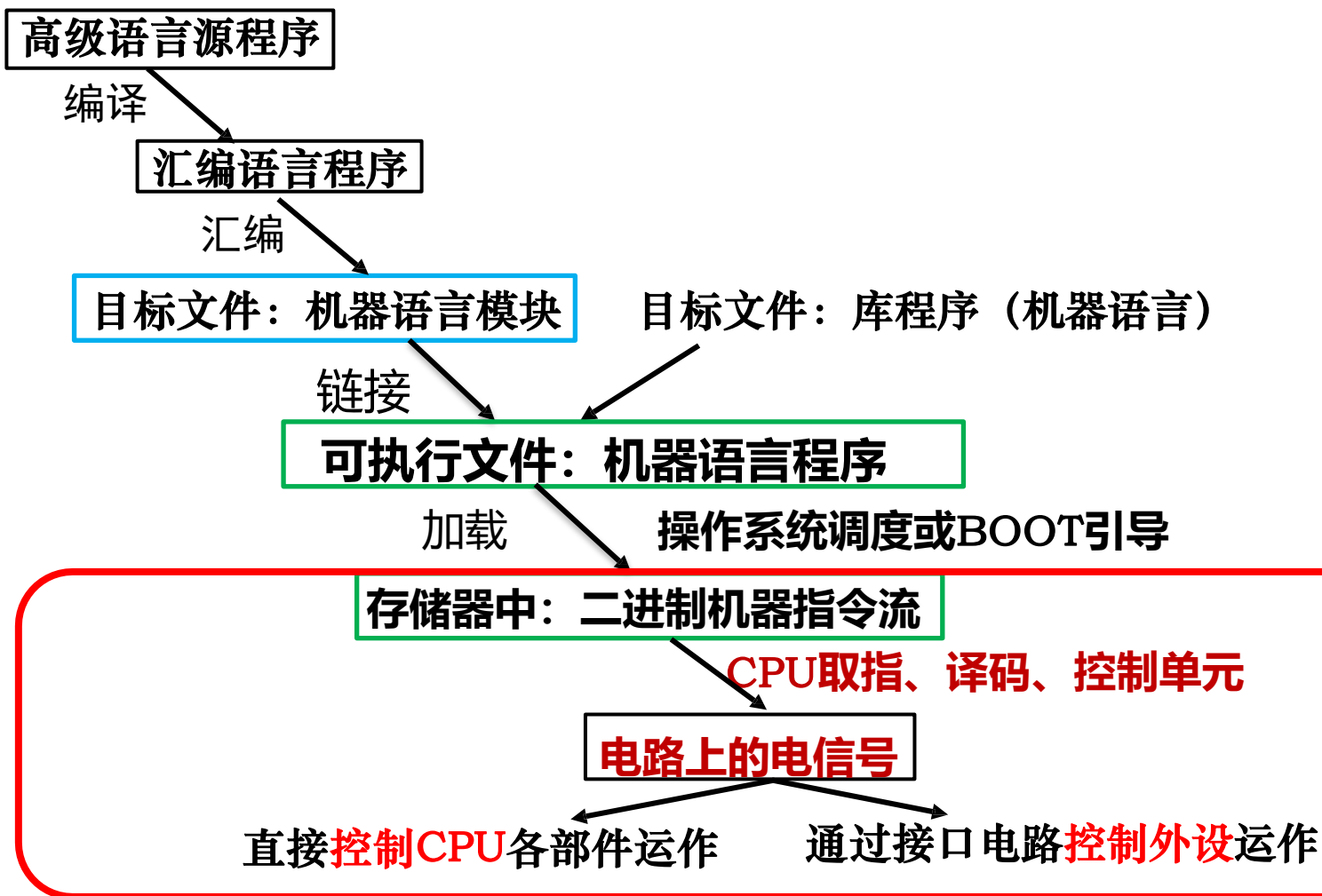
汇编

二进制机器语言

```
00000000101000100000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
101011100001001000000000000000000
101011011110001000000000000000100
0000001111100000000000000000001000
```

汇编语言代码(MIPS指令集), v数组地址存放在\$4中, k存放在\$5中

从高级语言到硬件语言



从高级语言到硬件语言

- 计算机内部工作过程：逐条执行加载到内存中的二进制机器指令流的过程。
- 一条指令的执行过程可简单地分为两个操作阶段：
 - **取指阶段**，CPU从内存中读取指令，程序计数器保存要被取出的下一条指令的地址，除非遇到跳转指令等情况，否则，PC一般都是在每次取指后加上一个增量（当前指令的字节数）；
 - **执行阶段**，对取出的指令先译码，解释指令的功能，然后执行译码好的指令，这期间可能会读写存储器或端口来获取操作数或者存放计算结果。
- 程序的执行过程就是周期性和重复性地进行取指令和执行指令两个操作。

1.5 性能评价

如何理解 “一台计算机性能比另一台好” 的含义？

先参考民航客机的 “性能”：载客量、续航能力、巡航速度、运载能力



Airplane	Passenger capacity	Cruising range (miles)	Cruising speed (m.p.h.)	Passenger throughput (passengers x m.p.h.)
Boeing 777	375	4630	610	228,750
Boeing 747	470	4150	610	286,700
BAC/Sud Concorde	132	4000	1350	178,200
Douglas DC-8-50	146	8720	544	79,424

■ Time of Boeing 747 vs. Concorde?

巡航速度上Concorde比Boeing 747快: $1350/610=2.2$ 倍 Concorde的性能更好!

■ Throughput of Boeing 747 vs. Concorde?

运载能力上Boeing 747比Concorde大: $286,700/178,200=1.6$ 倍
Boeing 747的性能更好!

不同的性能评价标准会导致不同的结论!

性能评价指标

□ 衡量计算机性能的基本指标

- **响应时间**(Response Time): 计算机完成任务需要的总时间, 包括CPU执行时间(Execution Time)、访问存储时间, I/O时间和操作系统开销。
- **吞吐率**(Throughput): 也称为带宽(Bandwidth), 单位时间内所完成的任务总量
- **指令执行速度**(MIPS、MFLOPS)

性能的比较

□ 哪台计算机更快？两种不同的性能评价标准

- 响应时间

- 吞吐率

□ 不同应用场合用户关心的性能不同

- 吞吐率高的场合: 多媒体应用(音/视频播放要流畅)

- 响应时间短的场合: 事务处理系统(响应速度要快)

- 吞吐率高且响应时间短的场合: 文件服务器、Web服务器等

性能的比较

- 度量计算机性能最可靠的指标——程序的执行时间
 - 完成同样工作量所需时间最短的性能最好
 - 处理器同时处理多个程序，用户感觉到的程序执行时间并不仅仅是程序的CPU执行时间
- 系统响应时间：用户感受到的
 - CPU执行时间（CPU时间）：CPU上花费的时间
 - 用户CPU时间：CPU性能，用来运行用户程序的时间
 - 系统CPU时间：系统性能，操作系统为执行用户程序花费的时间
 - 其他时间：包括I/O操作和运行其他程序的时间

性能的比较

□ 希望执行时间最小，计算机性能度量标准：

$$\text{性能} = 1 / \text{执行时间}$$

若说计算机A比B性能好，即A的执行时间比B短

例：相对性能计算

如果计算机A运行一个程序需要10秒，计算机B运行同样的程序需要15秒，那么计算机A的性能是计算机B的几倍？

解：A的性能是B的1.5倍。

CPU性能及其因素

□时钟周期 (Clock cycle) & 时钟频率(Clock rate)

- 计算机有一个固定频率的硬件时钟，决定各种硬件事件发生和执行的时间和顺序
- 时钟周期：时钟的时间间隔
- 时钟频率：时钟周期的倒数

例：时钟周期为250ps，对应的时钟频率为多少？

时钟频率 = $1/\text{时钟周期} = 1/250\text{ps} = 4\text{GHz}$

一个程序的CPU时间 = 程序的CPU时钟周期数 $\times$ 时钟周期时间
= 程序的CPU时钟周期数 $\div$ 时钟频率

一个程序在一台时钟频率4GHz的计算机A上运行10秒，现在我们要设计一台时钟频率更高的计算机B，将该程序在计算机B上的执行时间降低为6秒。

提高时钟频率将影响CPU其他方面的设计，使得在计算机B上运行该程序的时钟周期数增长为1.2倍。请分析计算机B的时钟频率？

	机器A	机器B
CPU时钟频率	4GHz	8GHz?
程序运行的时钟周期数	I	$I \times 1.2$ ↑
执行时间	10秒	6秒 ↓

$$\therefore \text{CPU时间}_A = \text{CPU时钟周期数}_A / \text{时钟频率}_A$$

$$\therefore \text{CPU时钟周期数}_A = 10 \text{ sec} \times 4 \text{ GHz}$$

$$\begin{aligned} \text{时钟频率}_B &= \text{CPU时钟周期数}_B / \text{CPU时间}_B = 1.2 \times 10 \text{ sec} \times 4 \text{ GHz} / 6 \text{ sec} \\ &= 8 \text{ GHz} \end{aligned}$$

指令性能

□ **程序的执行时间** = 程序的CUP时钟周期数 × 时钟周期时间
= 程序的CPU时钟周期数 ÷ 时钟频率

程序的CPU时钟周期数 = 程序的指令数 × 每条指令的平均时钟周期数

CPI (clock Cycles Per Instruction)

一个程序的CPU时间 = 指令数 × CPI × 时钟周期时间

IPC: (Instruction Per clock Cycles) 每个时钟周期的指令数

MIPS: 每秒百万条指令 = 指令数 / (执行时间 × 10⁶)

性能指标	测量单位
程序的CPU执行时间	程序执行的秒数
指令数	程序执行的指令个数
CPI	每条指令的平均时钟周期个数
时钟周期时间	每个时钟周期的秒数

假设有相同指令集的两种不同实现方式：

计算机A的时钟周期为250ps，对某程序的CPI为2.0

计算机B的时钟周期为500ps，对同样程序的CPI为1.2

请问：对于该程序，请问哪台计算机执行的速度更快？快多少？

一个程序的CPU时间 = 指令数 × CPI × 时钟周期时间

相同指令集对于同一个程序，其指令序列一样，指令数相同

$$\begin{aligned}\text{CPU时间}_A &= \text{指令数} \times \text{CPI}_A \times \text{时钟周期时间}_A \\ &= I \times 2.0 \times 250\text{ps} \\ &= I \times 500\text{ps}\end{aligned}$$

$$\begin{aligned}\text{CPU时间}_B &= \text{指令数} \times \text{CPI}_B \times \text{时钟周期时间}_B \\ &= I \times 1.2 \times 500\text{ps} \\ &= I \times 600\text{ps}\end{aligned}$$

$$\text{CPU性能}_A / \text{性能}_B = \text{执行时间}_B / \text{执行时间}_A = (I \times 600\text{ps}) / (I \times 500\text{ps}) = 1.2$$

一个编译器设计者试图在两个代码序列之间进行选择，硬件设计者给出了如下数据：

	CPI for this instruction class		
	A	B	C
CPI	1	2	3

对于某条高级语言语句的实现，两个代码序列所需的指令数量如下：

Code sequence	Instruction counts for instruction class		
	A	B	C
1	2	1	2
2	4	1	1

哪个代码序列执行的指令数更多？哪个执行速度更快？每个代码序列的CPI是多少？

解： 序列1 有5条指令；序列2 有 6条

序列1 需 $2 \times 1 + 1 \times 2 + 2 \times 3 = 10$ cycles

序列2 需 $4 \times 1 + 1 \times 2 + 1 \times 3 = 9$ cycles

序列1 的 $CPI = 10/5 = 2$ ；序列2 的 $CPI = 9/6 = 1.5$

虽然序列2的指令条数多，
但速度却更快！

一个程序的CPU时间 = 指令数/程序 × CPI × 时钟周期时间

性能指标
指令数
CPI
时钟周期时间

三个因素如何相关?

- 改变指令集架构以减少指令条数可能会使时钟周期变长
- CPI的减少可能会增加时钟周期的长度
- 缩短时钟周期可能会增加CPI
- 即使是在同一台机器上解决同一个问题，最少指令条数的程序不一定执行的最快

CPI对于不同应用程序是不同的，甚至对于相同指令集的不同实现方式也是不同的。

评价性能时，必须在各个因素进行权衡！

决定程序性能的主要因素

- 程序中使用的算法——涉及数据结构、算法设计
- 程序并翻译成机器指令的软件——涉及编译原理
- 计算机各部件执行效率——涉及计算机体系结构、操作系统

理解程序性能

名称	影响因素	如何影响
算法	指令数、CPI	算法决定源程序的指令条数，因此决定了处理器执行的指令条数。算法包括的慢速或快速指令的数目不同会影响CPI。
编程语言	指令数、CPI	编程语言中的语句被翻译成CPU指令，决定了指令数和指令类型，也可能影响CPI。
编译器	指令数、CPI	编译器决定了从源语言语句到机器指令的翻译，对指令数及CPI均有影响。
指令集体系结构	指令数、时钟频率、CPI	指令集的结构对完成某功能需要的指令数、CPI及CPU时钟频率均有影响。

性能的综合评价

问题：如果用一组基准程序在不同的机器上测出了运行时间，那么如何综合评价机器的性能呢？

例： 程序 1： 在计算机A上执行时间1秒,在计算机B上执行时间10秒

程序 2： 在计算机A上执行时间10秒,在计算机B上执行时间1秒

会得出什么样的结论？

■ 对于程序1，A 的性能是B的10倍

■ 对于程序2，B 的性能是A的10倍

无法比较A和B的
好坏，须用一个
综合的值来表示

□ 一种简单的综合评价指标——总执行时间

总执行时间 = 运行时间_{程序1} × 出现频率_{程序1} + 运行时间_{程序2} × 出现频率_{程序2} + ...

SPEC CPU BENCHMARK

- 基准测试程序集是专门进行性能评价的一组程序
 - 用户希望基准测试程序能够预测机器在运行实际负载时的性能

SPEC INTC2006 基准程序在 Intel Core i7 920 上的运行结果

描述	名称	指令数目 ($\times 10^9$)	CPI	时钟周期时间 ($\times 10^{-9}$ 秒)	执行时间 (秒)	参考时间 (秒)	Spec 分值
字符串处理解释程序	perl	2 252	0.60	0.376	508	9 770	19.2
块排序压缩	bzip2	2 390	0.70	0.376	629	9 650	15.4
GNU C 编译器	gcc	794	1.20	0.376	358	8 050	22.5
组合优化	mcf	221	2.66	0.376	221	9 120	41.2
go 游戏 (人工智能)	go	1 274	1.10	0.376	527	10 490	19.9
围棋游戏 (人工智能)	hmmer	2 616	0.60	0.376	590	9 330	15.8
基因序列搜索	sjeng	1 948	0.80	0.376	586	12 100	20.7
量子计算机仿真	libquantum	659	0.44	0.376	109	20 720	190.0
视频压缩	h264avc	3 793	0.50	0.376	713	22 130	31.0
离散事件仿真库	omnetpp	367	2.10	0.376	290	6 250	21.5
游戏/寻找路径	astar	1 250	1.00	0.376	470	7 020	14.9
XML 解析	xalancbmk	1 045	0.70	0.376	275	6 900	25.1
几何平均	—	—	—	—	—	—	25.7

理解程序性能

例：假设某个程序在某台计算机上运行时所需的时间是100秒，其中80秒是用来执行乘法操作。如果希望使该程序的速度提高到原来的5倍，乘法部件的速度应该是原来的多少倍呢？若其中90秒用来执行乘法操作，结果又会如何？

Amdahl定律：

$$\text{改进后的执行时间} = \frac{\text{受改进影响部分的执行时间}}{\text{改进提高的倍数}} + \text{不受影响的执行时间}$$

因为程序执行速度提高到原来的5倍，故新的执行时间应该是 $100/5=20$ 秒

$$20\text{秒} = 80\text{秒}/n + (100-80)\text{秒}, \quad 0 = 80\text{秒}/n$$

结论：如果乘法只是占到总计算量的80%的话，无论对乘法部件做何种改进，系统性能都不可能提高到原来的5倍。

$$20\text{秒} = 90\text{秒}/n + (100-90)\text{秒}, \quad 10 = 90\text{秒}/n, \quad n = 9$$

乘法部件的速度需要达到原来的9倍。

硬件设计的基本策略：让最常用的部件执行得更快

1.6 集成电路的功耗与成本

□ 集成电路的功耗：大规模集成电路以CMOS电路为主

□ 动态功耗：CMOS中主要的能量消耗

■ 动态功耗主要是指晶体管动态翻转时消耗的能量

$$\text{功耗} \propto 1/2 \times \text{负载电容} \times \text{电压}^2 \times \text{开关频率}$$

□ 负载电容是晶体管扇出数和所用工艺的函数

□ 开关频率是时钟频率的函数

■ 减少芯片动态功耗的方法

□ 降低时钟频率

□ 降低工作电压 (从5V降低到1.5V)

□ 但是，低电压使晶体管的漏电流问题更加突出，大约30% ~ 40%的芯片功耗是由于漏电流引起的

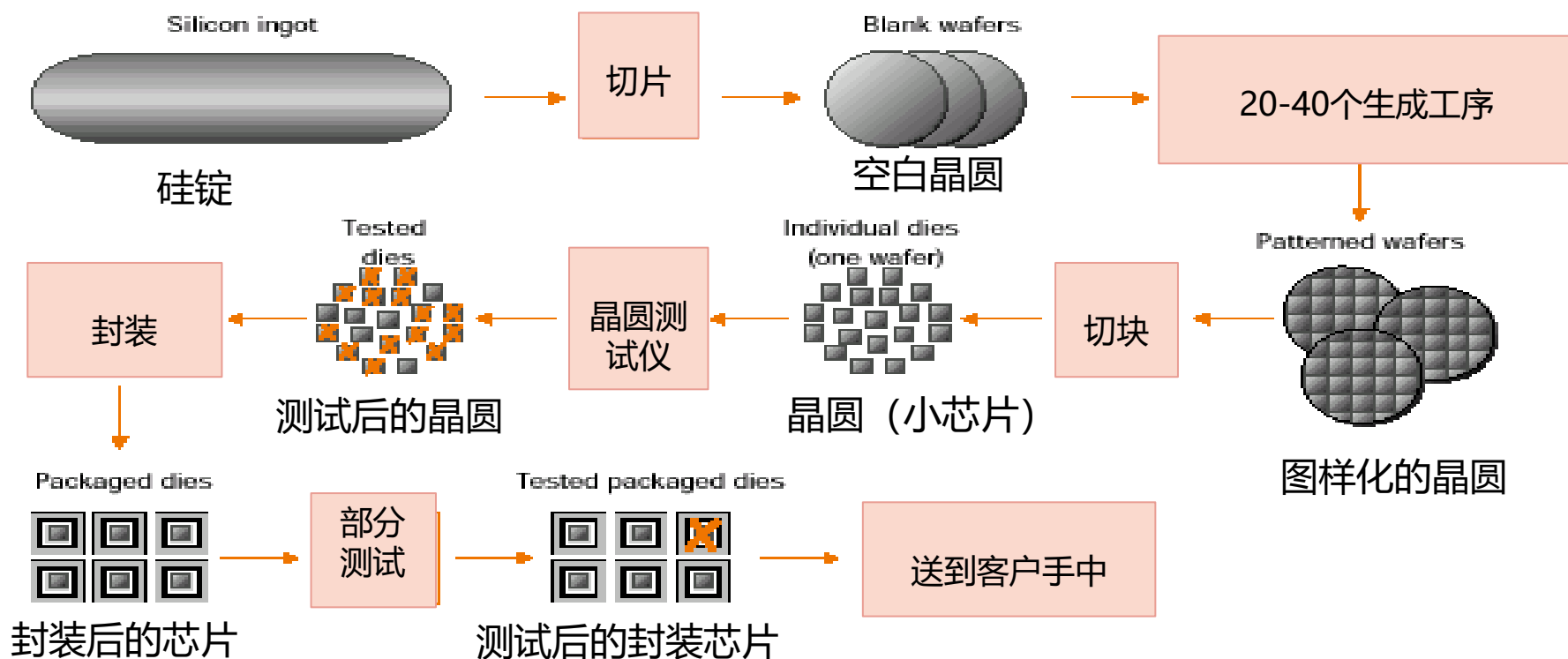
□ 静态功耗：即使芯片内晶体管关闭，仍有漏电流发生

$$\text{静态功耗} \propto \text{电压} \times \text{漏电流}$$

■ 即使晶体管总是关闭，增加晶体管的数量也会增加功耗

集成电路的成本

□ 芯片的制造过程



封装：将芯片固定在塑胶或陶瓷基座上，把芯片上蚀刻出来的引线与基座底部伸出的引脚连接，盖上盖板并封焊成芯片

集成电路成本

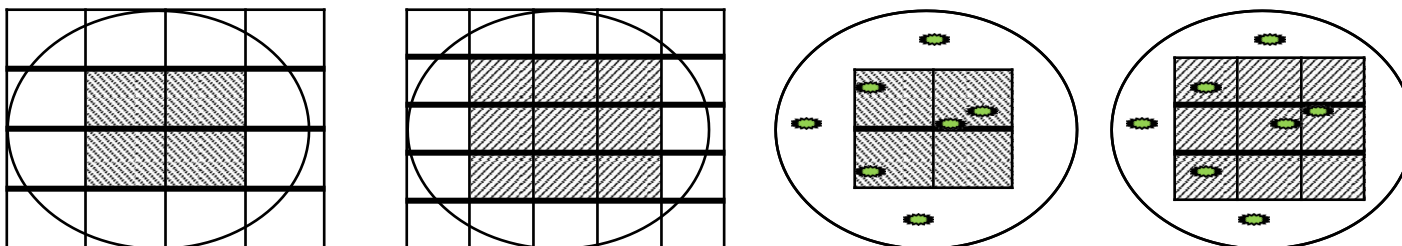
□与芯片成本有关的因素

- 晶圆价格
- 晶圆所含芯片数
- 芯片合格率

每芯片的价格 = 每晶圆的价格 / (每晶圆的芯片数 × 成品率)

每晶圆的芯片数 ≈ 晶圆面积 / 芯片面积

芯片合格率 = $1 / (1 + (\text{单位面积的瑕疵数} \times \text{芯片面积} / 2))^2$



$$a. \frac{\text{时钟频率}}{CPI}$$

$$P_1: \frac{3 \times 10^9}{1.5} = 2 \times 10^9 \quad P_2: 2.5 \times 10^9 \quad P_3: \frac{4}{2.2} \times 10^9$$

练习

$$b. P_1 \text{ 时钟周期数 } n = 10 \times 3 \times 10^9$$

$$P_2: 10 \times 2.5 \times 10^9$$

1.5 [4] <1.6> 有 3 种不同的处理器 P1、P2 和 P3 执行同样的指令集，P1 的时钟频率为 3GHz，CPI 为 1.5；P2 的时钟频率为 2.5GHz，CPI 为 1.0；P3 的时钟频率为 4GHz，CPI 为 2.2。

a. 以每秒钟执行的指令数目为标准，哪个处理器性能最高？

$$P_{\text{性能}} = \frac{1}{\text{执行时间}} = \frac{1}{CPI \times T_{\text{clock}} \times N} = \frac{f}{CPI \times N}$$

b. 如果每个处理器执行一个程序都花费 10 秒钟时间，求它们的时钟周期数和指令数。

c. 我们试图把执行时间减少 30%，但这会引起 CPI 增加 20%。问：时钟频率应该是多少才能达到时间减少 30% 的目的？

1.6 [20] <1.6> 同一个指令集体系结构有两种不同的实现方式。根据 CPI 的不同将指令分成 4 类 (A、B、C 和 D)，P1 的时钟频率为 2.5GHz，CPI 分别为 1、2、3 和 3；P2 时钟频率为 3GHz，CPI 分别为 2、2、2 和 2。

给定一个程序，有 1.0×10^6 条动态指令，按如下比例分为 4 类：A，10%；B，20%；C，50%；D，20%。

a. 每种实现方式总的 CPI 是多少？

b. 计算两种情况下的时钟周期数。(个数)

$$b. P_1: T_{\text{clock}} = \frac{1}{f} = \frac{1}{2.5 \times 10^9}$$

$$a. P_1: 1 \times 0.1 + 2 \times 0.2 + 3 \times 0.5 + 3 \times 0.2 = 2.1 + 0.4 + 0.1 = 2.6$$

$$P_2: 2$$

$$P_2: \frac{1}{3 \times 10^9}$$

练习

- 1.5** [4] <1.6> 有 3 种不同的处理器 P1、P2 和 P3 执行同样的指令集，P1 的时钟频率为 3GHz，CPI 为 1.5；P2 的时钟频率为 2.5GHz，CPI 为 1.0；P3 的时钟频率为 4GHz，CPI 为 2.2。
- a. 以每秒钟执行的指令数目为标准，哪个处理器性能最高？
 - b. 如果每个处理器执行一个程序都花费 10 秒钟时间，求它们的时钟周期数和指令数。
 - c. 我们试图把执行时间减少 30%，但这会引起 CPI 增加 20%。问：时钟频率应该是多少才能达到时间减少 30% 的目的？
- 1.6** [20] <1.6> 同一个指令集体系结构有两种不同的实现方式。根据 CPI 的不同将指令分成 4 类 (A、B、C 和 D)，P1 的时钟频率为 2.5GHz，CPI 分别为 1、2、3 和 3；P2 时钟频率为 3GHz，CPI 分别为 2、2、2 和 2。
- 给定一个程序，有 1.0×10^6 条动态指令，按如下比例分为 4 类：A，10%；B，20%；C，50%；D，20%。
- a. 每种实现方式总的 CPI 是多少？
 - b. 计算两种情况下的时钟周期。